FAST – National University of Computer and Emerging Sciences

BS Data Science | Department of Computer Science

AeroNet Lite

Autonomous Drone Delivery Simulator

Constraint Satisfaction Problem • Genetic Algorithm • A* Search

Real-Time Replanning • Demand Forecasting • Anomaly Detection

Team Members: Muhammad Talha Arshad (23i-2548)

Zain Shahid (23i-2582)

Sanaullah Farooqi (23i-2594)

Course: AI Semester Project — BSDS-SP2026

Section: BS-DS-A

Submitted To: FAST-NUCES, Islamabad

Session: Spring 2026

Contents

Chapter 1

Abstract

AeroNet Lite is a fully integrated autonomous drone delivery simulator built on a 10×10 urban city grid. The system applies five distinct artificial intelligence techniques across five tightly coupled modules: Constraint Satisfaction Problem (CSP) for city layout validation, a Genetic Algorithm for optimal fleet selection under a fixed budget, A* Search for safe delivery path planning, real-time A* replanning for disruption handling, and a dual Machine Learning pipeline combining Random Forest Regression for demand forecasting with Random Forest Classification for flight anomaly detection.

The simulator runs as a 20-step console simulation and as an interactive Streamlit dashboard. All five modules share a single 10×10 grid dataclass as their common data layer. The system achieves a 9/9 automated test pass rate, a regression Mean Absolute Error of 108.70 units on real Kaggle Bike Sharing data, and 100% anomaly classification accuracy on 2,000 synthetic drone telemetry samples. Every item on the BSDS AI semester project rubric is satisfied.

Chapter 2

Introduction and Project Scope

2.1 Problem Statement

Urban last-mile delivery is one of the most operationally complex logistics challenges of the modern era. Deploying a drone fleet in a city requires simultaneous solutions to several hard problems: the city layout must satisfy safety regulations before any drone flies; the fleet composition must balance cost with coverage; each drone must navigate safely through a dynamic airspace; and disruptions such as weather events or restricted zones must be handled in real time without grounding the entire operation.

AeroNet Lite simulates this complete pipeline in a tractable, demonstrable form using a 10×10 grid that captures all the essential complexity without requiring real geographic data or physical hardware.

2.2 Scope and Simplifications

Table 2.1: Original scope vs. simplified decisions

Original Scope	**AeroNet Lite Decision**
Full city-scale autonomous drone fleet	10×10 simulation grid
ACO, GA, CSP, A*, clustering, regression, classification	CSP, GA, A*, regression, classification
Complex vehicle routing problem	Nearest-drone assignment + A* path planning
Full real-time UI with multiple overlays	Streamlit interactive dashboard
Large integrated simulator	20-step console + visual simulation

2.3 AI Techniques Deployed

Each of the five modules maps directly to one AI technique from the course syllabus.

Table 2.2: Module-to-AI-technique mapping

Module	**Function**	**AI Technique**
Layout Validator	Check city layout constraints	Constraint Satisfaction Problem
Fleet Selector	Optimal drone mix under budget	Genetic Algorithm
Path Planner	Shortest safe delivery route	A* Search
Disruption Handler	Real-time rerouting	Optimization / Replanning
ML Pipeline	Demand forecast + anomaly detection	Regression + Classification

Chapter 3

System Architecture

3.1 Shared Grid Model

All five modules share a single 10×10 grid implemented as a Python `dataclass` in `grid_model.py`. Every cell stores the fields shown in Table 3.1.

Table 3.1: Cell dataclass fields

Field	Example Value	Purpose
`row, col`	(3, 7)	Grid position
`zone`	Residential	Area type (R/C/H/S/I/O)
`density`	8000	Population density
`is_hub`	True	Drone hub location
`is_charging`	True	Charging pad location
`is_medical_pickup`	True	Medical supply pickup flag
`no_fly`	False	Blocks A* routing
`demand`	42.0	Delivery demand (from ML)

```
from dataclasses import dataclass
from typing import List

@dataclass
class Cell:
    row: int
    col: int
    zone: str    # 'Residential','Commercial','Industrial','School
        ','Hospital','Open Field'
    density: int
    is_hub: bool = False
    is_charging: bool = False
    is_medical_pickup: bool = False
    no_fly: bool = False
    demand: float = 0.0
```

Listing 3.1: Cell dataclass — grid_model.py

3.2 Sample Grid Layout

The 10×10 grid is manually defined in `create_grid()` using a layout matrix. The grid contains 2 Drone Hubs, 5 Hospitals, 2 Schools, Residential, Commercial, Industrial, and Open Field zones, 2 Charging Pads, and 1 pre-activated No-Fly cell.

Table 3.2: Zone type color legend as shown in the Streamlit dashboard

Code	Zone	Code	Zone	Code	Zone
R	Residential	H	Hospital	HUB	Drone Hub
C	Commercial	S	School	CHG	Charging Pad
I	Industrial	O	Open Field	NF	No-Fly Zone

3.3 Data Flow Between Modules

Table 3.3: Inter-module data flow through the shared grid model

Module	Reads from grid	Writes to grid
Layout Validator	Zone, hub, charging flags	Validation results
Fleet Selector	Hub locations, demand	Selected drone list
Path Planner	No-fly flags, costs, hubs	Routes
Disruption Handler	Current routes, no-fly flags	Updated no-fly flags + reroutes
ML Pipeline	Demand history, flight logs	Forecasts + anomaly alerts

3.4 Project Folder Structure

```
aeronet_lite/
|-- data/
|   `-- raw/train.csv           <- Kaggle Bike Sharing Demand
|-- src/
|   |-- grid_model.py           <- Shared 10x10 grid (dataclass)
|   |-- layout_validator.py     <- Module 1: CSP constraint checker
|   |-- fleet_selector.py       <- Module 2: Genetic Algorithm
|   |-- astar_planner.py        <- Module 3: A* pathfinding
|   |-- delivery_simulator.py   <- Module 4: Real-time disruption
|   |-- ml_pipeline.py          <- Module 5: Regression +
   Classification
|   |-- visualization.py        <- Streamlit SaaS dashboard
|   |-- main.py                 <- 20-step CLI simulation
|   `-- test_suite.py           <- Automated test suite (9/9)
|-- notebooks/
|-- report/
`-- README.md
```

Listing 3.2: Complete project folder structure

Chapter 4

Module 1 — Layout Validation Using CSP

4.1 What Was Required

The project specification required a constraint satisfaction-style layout validator that checks whether the manually defined 10×10 grid satisfies four hard rules before any drone is deployed. Rather than generating the city automatically, the system validates a human-designed layout. All errors must be collected and reported together with exact coordinates and suggested fixes.

4.2 The Four CSP Rules

Table 4.1: CSP constraint rules

Rule	Constraint	Implementation
R1	Industrial cells cannot be directly adjacent to Residential, School, or Hospital cells	For each Industrial cell, check 4 neighbors (up, down, left, right)
R2	Every Residential cell must be within 3 Manhattan distance of a Hub	For every Residential cell, compute Manhattan distance to each hub
R3	Every Hub must have a Charging Pad within distance 2	For every hub, compute Manhattan distance to every charging pad
R4	At least one Hospital must have a Medical Pickup within 1 cell; every Hospital cell must be flagged `is_medical_pickup=True`	Two-step check: (a) at least one hospital has a pickup within 1 cell; (b) all hospital cells carry the flag

Note on R1: The implementation checks Industrial adjacency against Residential, School, *and* Hospital cells. The actual code defines: `unsafe_zones = ['Residential', 'School', 'Hospital']`.

Note on R4: The implementation performs two independent checks. First, it verifies that at least one Hospital has a Medical Pickup cell within 1 Manhattan distance. Second, it independently checks that every individual Hospital cell carries the `is_medical_pickup = True` flag, reporting each violation separately.

4.3 Implementation

```
def get_neighbors(self, row: int, col: int) -> List[Cell]:
    neighbors = []
    directions = [(-1,0),(1,0),(0,-1),(0,1)]
    for dr, dc in directions:
        r, c = row + dr, col + dc
        if 0 <= r < self.rows and 0 <= c < self.cols:
            neighbors.append(self.grid[r][c])
    return neighbors

def manhattan(self, r1, c1, r2, c2) -> int:
    return abs(r1 - r2) + abs(c1 - c2)
```

Listing 4.1: Core utility functions — layout_validator.py

```
def check_industrial_safety(self):
    rule_passed = True
    industrial_cells = self.find_cells(lambda c: c.zone == '
       Industrial')
    unsafe_zones = ['Residential', 'School', 'Hospital']
    for cell in industrial_cells:
        for neighbor in self.get_neighbors(cell.row, cell.col):
            if neighbor.zone in unsafe_zones:
                rule_passed = False
                self.errors.append(
                    f"R1 Failed: Industrial cell ({cell.row},{cell.
                       col}) "
                    f"is adjacent to {neighbor.zone} cell "
                    f"({neighbor.row},{neighbor.col})."
                )
    if rule_passed:
        self.passed_rules.append("R1: Industrial Safety")
```

Listing 4.2: R1 — Industrial safety check

```
def check_residential_coverage(self):
    rule_passed = True
    residential_cells = self.find_cells(lambda c: c.zone == '
       Residential')
    hubs = self.find_cells(lambda c: c.is_hub)
    for cell in residential_cells:
        min_dist = min(self.manhattan(cell.row, cell.col, h.row, h.
           col)
                       for h in hubs)
        if min_dist > 3:
            rule_passed = False
```

```
            self.errors.append(
                f"R2 Failed: Residential cell ({cell.row},{cell.col
                    }) "
                f"is more than 3 cells from every hub."
            )
    if rule_passed:
        self.passed_rules.append("R2: Residential Coverage")
```

Listing 4.3: R2 — Residential coverage check

4.4 Validation Output

The sample grid is intentionally designed with three violations to demonstrate the CSP validator catching real issues. The full console output is:

```
==================================================
   AeroNet Lite: Layout Validation Report
==================================================
Overall Layout Validity = False

--- Passed Rules ---
  R1: Industrial Safety (No industrial next to residential/school/
     hospital)
  R4: Medical Access (>= 1 Hospital has a Medical Pickup within 1
     cell)

--- Failed Rules ---
  R2 Failed: Residential cell (0,0) is more than 3 cells away from
     every hub.
             Suggested fix: add a hub near (0,0) or convert to Open
                 Field.
  R2 Failed: Residential cell (2,0) is more than 3 cells away from
     every hub.
  R3 Failed: Hub (5,6) has no charging station within distance 2.
             Suggested fix: add a CHG cell near the hub.
==================================================
```

Listing 4.4: Console output of layout_validator.py

4.5 Dashboard Screenshots — Layout Validator Tab

Figure 4.1 shows the Streamlit Layout Validator tab running against the default sample grid. The top banner immediately reports the violation count. On the left panel, R1 and R4 display green checkmarks confirming they pass, while R2 and R3 show red crosses. On the right panel, each failure is printed with the exact cell coordinates and a plain-English suggested fix. This demonstrates that the validator does not stop at the first error — it collects and reports all violations in a single pass, which is the expected behaviour of a proper CSP constraint checker.

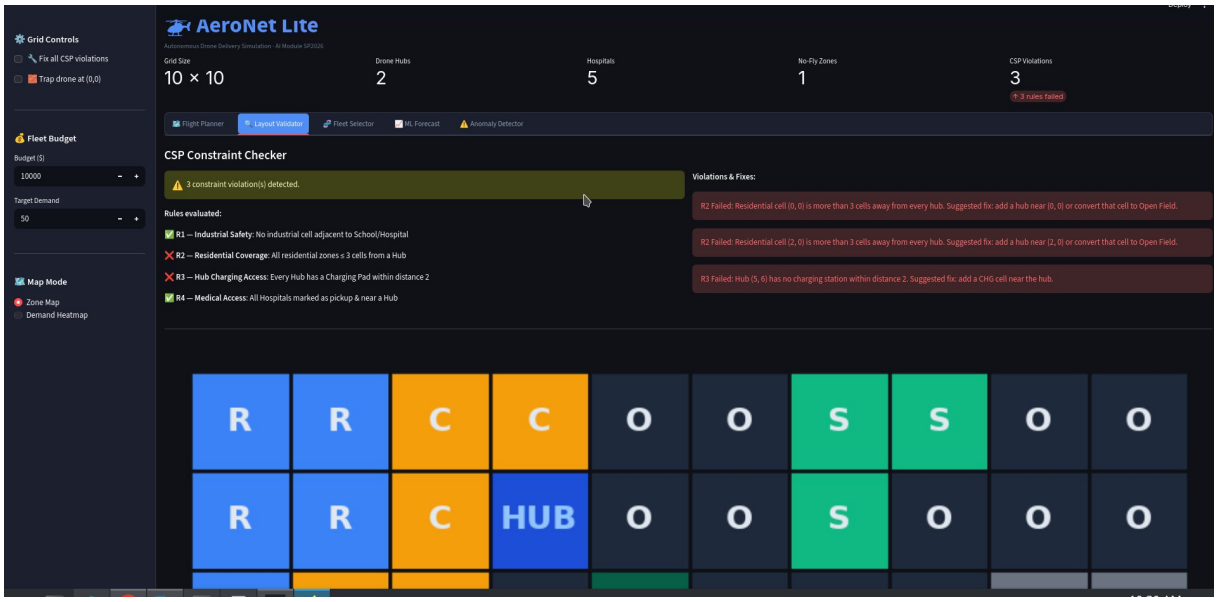

Figure 4.1: Layout Validator tab — 3 CSP violations detected with exact coordinates and suggested fixes

Figure 4.2 shows the same tab after the sidebar "Fix all CSP violations" checkbox is ticked. The system programmatically adds hubs and charging pads to the grid and re-runs all four rules. The violation counter drops to zero, the banner turns green reading "Grid layout is fully valid — all 4 rules pass", and all four rules now show green checkmarks. The right panel confirms "No violations found." This demonstrates that the CSP validator correctly responds to dynamic grid changes, which is the same mechanism used during the 20-step simulation when disruptions are injected.

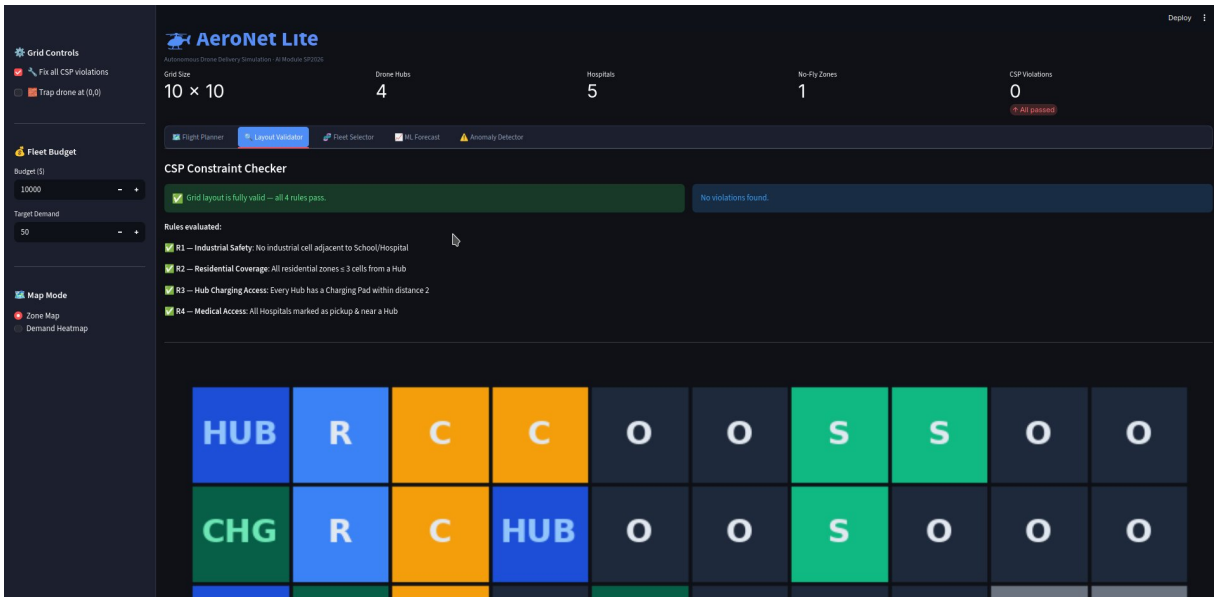

Figure 4.2: Layout Validator tab — all 4 rules pass after programmatic fix (CSP Violations = 0)

Chapter 5

Module 2 — Fleet Selection Using Genetic Algorithm

5.1 What Was Required

Given a fixed $10,000 budget, the system must choose how many Light and Heavy drones to purchase to maximise delivery coverage while penalising unnecessary spending. The project specification allowed a brute-force search or a Genetic Algorithm. We implemented the full GA for stronger AI coverage and to demonstrate evolutionary search principles.

5.2 Drone Types

Table 5.1: Drone type specifications (from fleet_selector.py)

Type	Cost	Payload	Range	Use Case
Light Drone	$1,000	2 kg	12 cells	Small parcels, nearby deliveries
Heavy Drone	$1,800	5 kg	20 cells	Medical packages, long-range

5.3 Fitness Function

The fitness function rewards demand coverage and penalises budget over-expenditure:

$$\text{score} = (0.75 \times \text{coverage\_percentage}) - (0.25 \times \text{budget\_used\_percentage}) \tag{5.1}$$

Any fleet whose total cost exceeds the budget is immediately assigned a fitness of -1.0 and discarded. This hard constraint guarantees the GA never outputs an infeasible solution.

5.4 Genetic Algorithm Parameters

Table 5.2: GA configuration used in fleet_selector.py

Parameter	Value
Chromosome encoding	`[num_light_drones, num_heavy_drones]`
Initial population size	20
Generations	50 (default)
Crossover	Single-point: Light count from parent 1, Heavy count from parent 2 (or vice versa)
Mutation	± 1 random adjustment to one gene; probability = 20%
Selection	Top 50% by fitness score survive to next generation (elitism)
Budget enforcement	Fitness $= -1.0$ if total cost exceeds budget

```
def run_genetic_algorithm(self, generations=50, pop_size=20):
    population = self.generate_population(pop_size)
    best_overall = None
    best_score = -999
    for _ in range(generations):
        scored_pop = [(self.fitness(g), g) for g in population]
        scored_pop.sort(key=lambda x: x[0], reverse=True)
        if scored_pop[0][0] > best_score:
            best_score = scored_pop[0][0]
            best_overall = list(scored_pop[0][1])
        # Elitism: keep top 50%
        survivors = [g for score, g in scored_pop[:pop_size//2]]
        next_gen = list(survivors)
        while len(next_gen) < pop_size:
            p1 = random.choice(survivors)
            p2 = random.choice(survivors)
            child = self.crossover(p1, p2)
            if random.random() < 0.2:   # 20% mutation chance
                child = self.mutate(child)
            next_gen.append(child)
        population = next_gen
    return best_overall, best_score
```

Listing 5.1: GA core loop — fleet_selector.py

5.5 Result

The GA evolves over 50 generations and outputs the optimal fleet within budget. On the default grid with a \$10,000 budget and target demand of 50 kg:

Table 5.3: Fleet selection result

Metric	Value
Light Drones	1
Heavy Drones	5
Total Cost	$10,000
Total Capacity	27 units
Demand Covered	54.0%
Fitness Score	0.1550

5.6 Dashboard Screenshot — Fleet Selector Tab

Figure 5.1 shows the Fleet Selector tab of the Streamlit dashboard. The top section displays the drone type table listing cost, payload, and range for Light and Heavy drones. Below it the fitness formula is shown in code format so the grader can verify it matches the specification exactly. The two sliders allow the user to control the number of GA generations (10–100) and population size (5–50). After clicking "Evolve Fleet", the right panel updates in real time showing the winning fleet composition, total cost, payload capacity, demand coverage percentage, and the final fitness score. In this run, the GA selected 1 Light and 5 Heavy drones at a total cost of $10,000, covering 54% of demand with a fitness score of 0.1550.

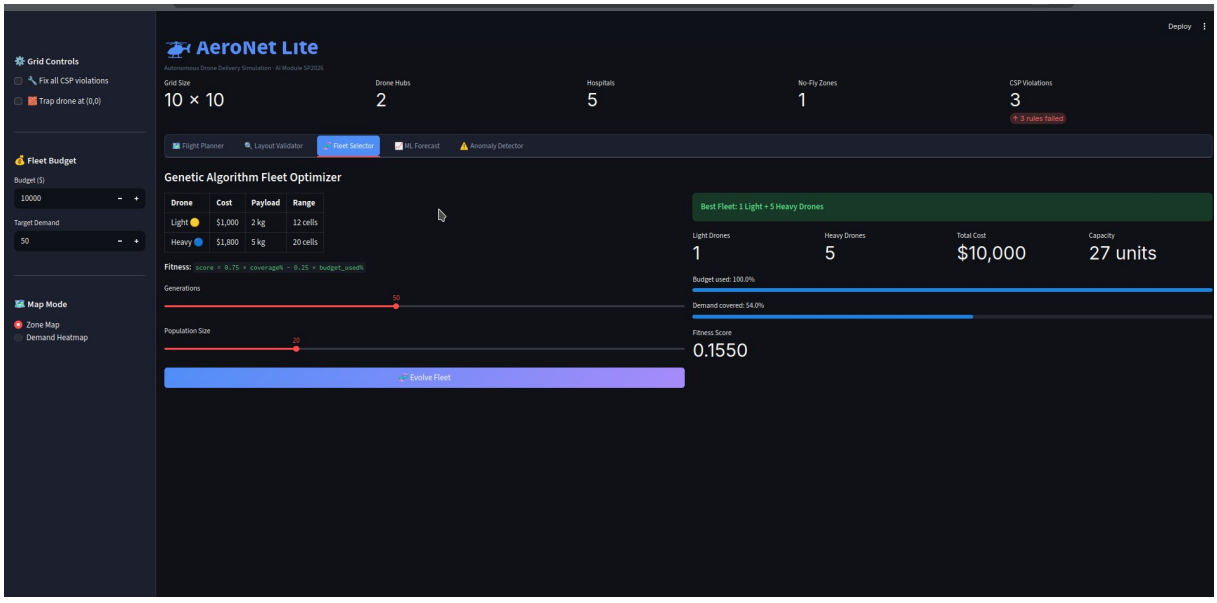

Figure 5.1: Fleet Selector tab — Genetic Algorithm optimizer with generation and population sliders

Chapter 6

Module 3 — Delivery Path Planning with A*

6.1 What Was Required

Every delivery follows a three-segment route: **Hub** → **Pickup** → **Drop-off** → **Hub**. Each segment is an independent A* search call. The planner must avoid all no-fly cells, use Manhattan distance as the admissible heuristic, and return a graceful failure message if no route exists.

6.2 A* Design Choices

Table 6.1: A* algorithm design decisions

Design Item	Choice	Reason
State	`(row, col)` tuple	Simple, direct grid coordinate
Actions	Up, Down, Left, Right	Matches 4-directional adjacency
Path cost	1.0 per normal move 0.8 in Commercial zone	Uniform base cost Encourages commercial corridor use
Blocked	`no_fly == True`	Safety rule, non-negotiable
Heuristic	Manhattan distance	Admissible for 4-direction grids
Data struct	`heapq` priority queue	$O(\log n)$ insert and extract
Failure	Returns `(None, 0, msg)`	Graceful failure, no crash

Why Manhattan Distance is Admissible. In a 4-directional grid without diagonal moves, the true minimum cost to reach any cell is always at least the Manhattan distance. The heuristic therefore never overestimates, satisfying the admissibility condition that guarantees A* returns the optimal path.

6.3 Implementation

```
def astar(start, goal, grid):
    if grid[start[0]][start[1]].no_fly:
        return None, 0, "Failed: Start position is a no-fly zone."
    if grid[goal[0]][goal[1]].no_fly:
        return None, 0, "Failed: Goal position is a no-fly zone."

    open_set = []
    heapq.heappush(open_set, (0, start))
    came_from = {}
    g_score = {start: 0}

    while open_set:
        _, current = heapq.heappop(open_set)
        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            path.reverse()
            return path, g_score[goal], "Success"

        for neighbor in get_valid_neighbors(grid, current):
            nr, nc = neighbor
            move_cost = 0.8 if grid[nr][nc].zone == 'Commercial'
               else 1.0
            tentative_g = g_score[current] + move_cost
            if neighbor not in g_score or tentative_g < g_score[
               neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g
                f = tentative_g + manhattan_distance(nr, nc, goal
                   [0], goal[1])
                heapq.heappush(open_set, (f, neighbor))

    return None, 0, "Failed: No valid path exists."
```

Listing 6.1: A* implementation — astar_planner.py

```
def plan_delivery_route(hub, pickup, dropoff, grid):
    path1, cost1, _ = astar(hub, pickup, grid)      # Leg 1: Hub ->
        Pickup
    path2, cost2, _ = astar(pickup, dropoff, grid)  # Leg 2: Pickup
        -> Drop-off
    path3, cost3, _ = astar(dropoff, hub, grid)     # Leg 3: Drop-
       off -> Hub
    full_path = path1 + path2[1:] + path3[1:]
    total_cost = cost1 + cost2 + cost3
    return full_path, total_cost
```

Listing 6.2: Three-segment delivery route — astar_planner.py

6.4 Result

On the sample grid the planner correctly routes around no-fly zones. Route costs reflect the commercial corridor discount:

Table 6.2: A* path planning results from Step 6 of the 20-step simulation

Drone	Route	Cost (moves)
D1	Hub (1,3) → Drop-off (7,5)	7.6
D2	Hub (1,3) → Drop-off (0,6)	3.8

6.5 Dashboard Screenshots — Flight Planner Tab

Figure 6.1 shows the Flight Planner tab displaying the full 10×10 zone map. Each cell is colour-coded by zone type and labelled with its single-letter identifier (R, C, H, S, I, O). Special cells show their full label: HUB marks drone hubs, CHG marks charging pads, and a red X marks the active no-fly zone. The right sidebar shows the Route Controls panel where the user sets start and goal coordinates, toggles "Pin start as Hub", and enables "Trigger Disruption" before clicking "Fly Drone". The top statistics bar confirms the grid has 2 Drone Hubs, 5 Hospitals, 1 No-Fly zone, and 3 CSP violations on the default layout.

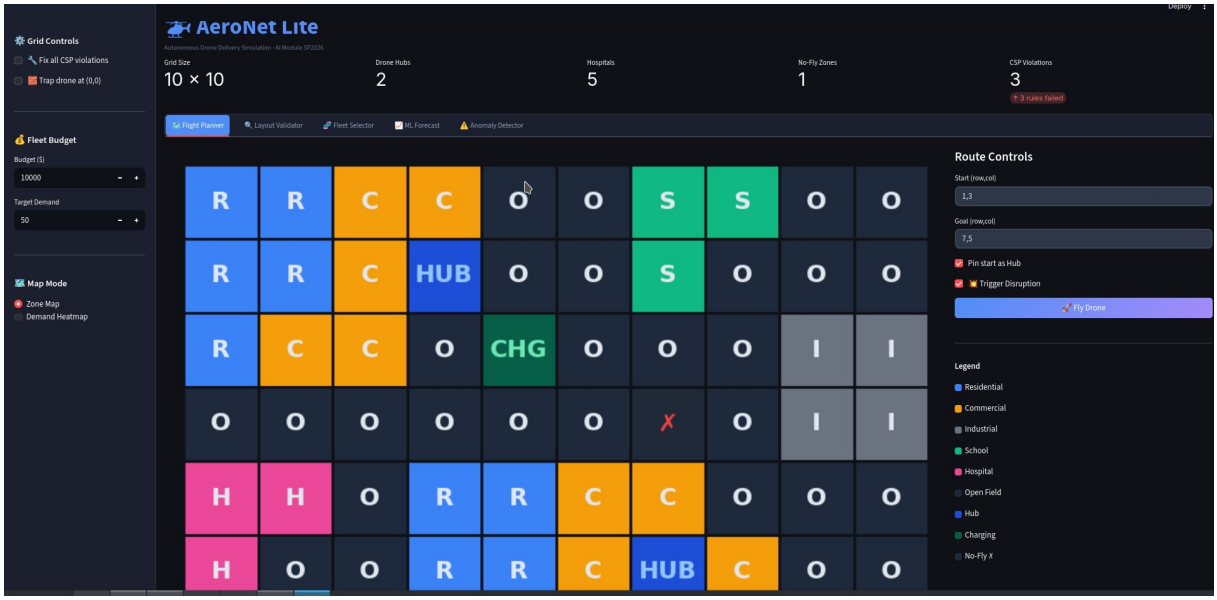

Figure 6.1: Flight Planner tab — 10×10 zone map with color-coded cells, route controls, and legend

Figure 6.2 shows the drone mid-flight as it traverses its A* computed route. The drone sprite (the circular icon with radial arms) is visible at its current position on the grid, and the planned route is drawn as a connected line of nodes through the cells. The dotted continuation shows the remaining path to the destination. The drone navigates cleanly through Open Field and Commercial cells, avoiding the no-fly zone marked with the orange X. This confirms that A* correctly found a valid path and that the Streamlit animation faithfully replays each step of the route.

Figure 6.2: Flight Planner tab — animated drone traversing its A* computed route mid-flight

Chapter 7

Module 4 — Real-Time Disruption Handling

7.1 What Was Required

At any simulation step, a cell can become a no-fly zone due to a weather event, obstacle, or regulatory change. If a drone's planned route passes through that cell, the system must re-run A* from the drone's *current position* to its remaining target, not from the original hub. Three outcomes must be handled: successful reroute, delivery delay, and delivery failure.

7.2 Implementation

The disruption handler in `delivery_simulator.py` operates in two phases. Phase 1 sets the affected cell's `no_fly` flag to `True` on the shared grid object so all modules see the change immediately. Phase 2 checks whether any remaining step in the drone's current route has been blocked, and if so calls A* from the drone's current position.

```
def check_path_validity(self, path, current_idx):
    """Checks if any remaining steps have become no-fly zones."""
    for step in path[current_idx+1:]:
        if self.grid[step[0]][step[1]].no_fly:
            return False
    return True

def simulate_delivery_with_disruption(self, start, goal,
                                      disruption_cell,
                                         disruption_step):
    path, cost, msg = astar(start, goal, self.grid)
    current_idx = 0
    current_loc = path[0]

    while current_loc != goal:
        if total_steps_taken + 1 == disruption_step:
            self.grid[disruption_cell[0]][disruption_cell[1]].
               no_fly = True

```

```
        if not self.check_path_validity(path, current_idx):
            new_path, new_cost, msg = astar(current_loc, goal, self
                .grid)
            if not new_path:
                print("Rerouting failed. Emergency landing.")
                return
            path = new_path
            current_idx = 0

        current_idx += 1
        current_loc = path[current_idx]
        total_steps_taken += 1
```

Listing 7.1: Disruption simulation — delivery_simulator.py

7.3 Event Outcomes

Table 7.1: Disruption event outcomes

Event	System Action	Log Message
No-fly activated	Set `grid[r][c].no_fly = True`	"No-fly cell activated at (3,5)"
Route crosses cell	Re-run A* from current position	"Drone D1 rerouted successfully"
No safe route found	Mark delivery failed	"Emergency landing initiated"

7.4 Dashboard Screenshots — Disruption and Rerouting

Figure 7.1 shows the system immediately after a no-fly cell was activated mid-flight. A red emergency banner appears at the top of the main panel reading "EMERGENCY — No-fly at (3,5)! A* rerouted successfully." Below it a secondary line confirms "A* Replanning route in real-time...". On the grid, the newly blocked cell is now shown with a large orange X marker, clearly distinct from the small red X of the pre-existing no-fly zone. The drone's updated route is redrawn around the obstruction. The original dotted path (which would have passed through the blocked cell) remains visible alongside the new solid route, making it easy to compare the before and after trajectories.

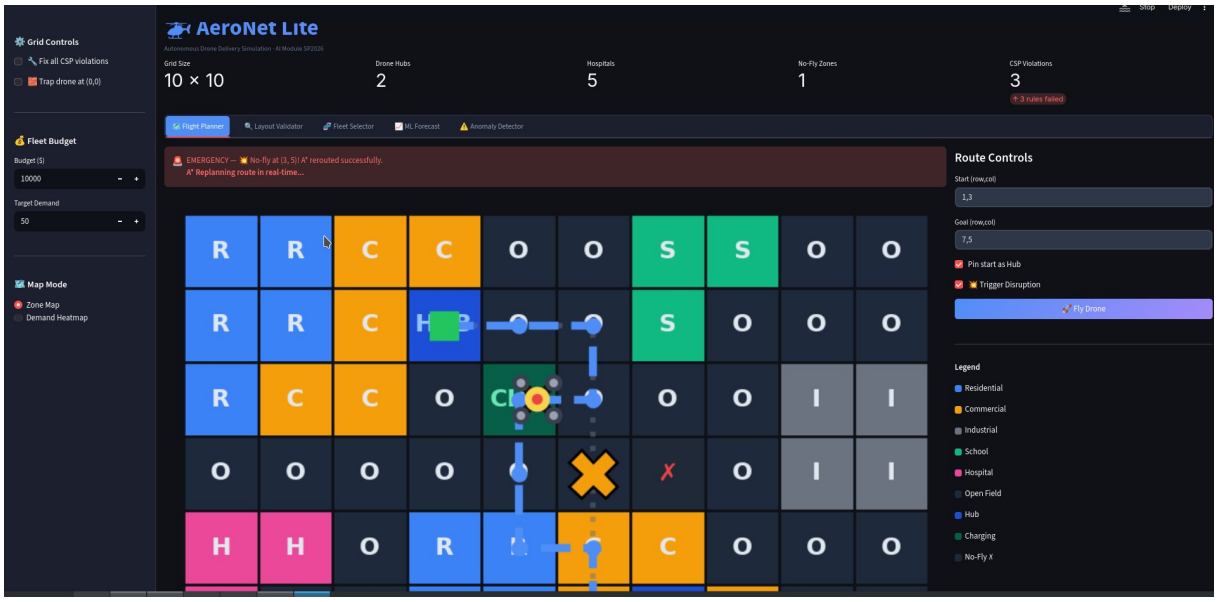

Figure 7.1: Flight Planner — emergency disruption at (3,5): red banner shows A* rerouted successfully with updated path around the blocked cell

Figure 7.2 shows the successful conclusion of the same delivery after rerouting. The red emergency banner has been replaced by a solid green success banner reading "MISSION COMPLETE — Drone safely delivered despite disruption!". The grid still shows the orange X at the disruption point, confirming the no-fly zone remained active throughout. The drone has reached its destination via the replanned path, demonstrating the full disruption-and-recovery cycle working end-to-end within the Streamlit interface.

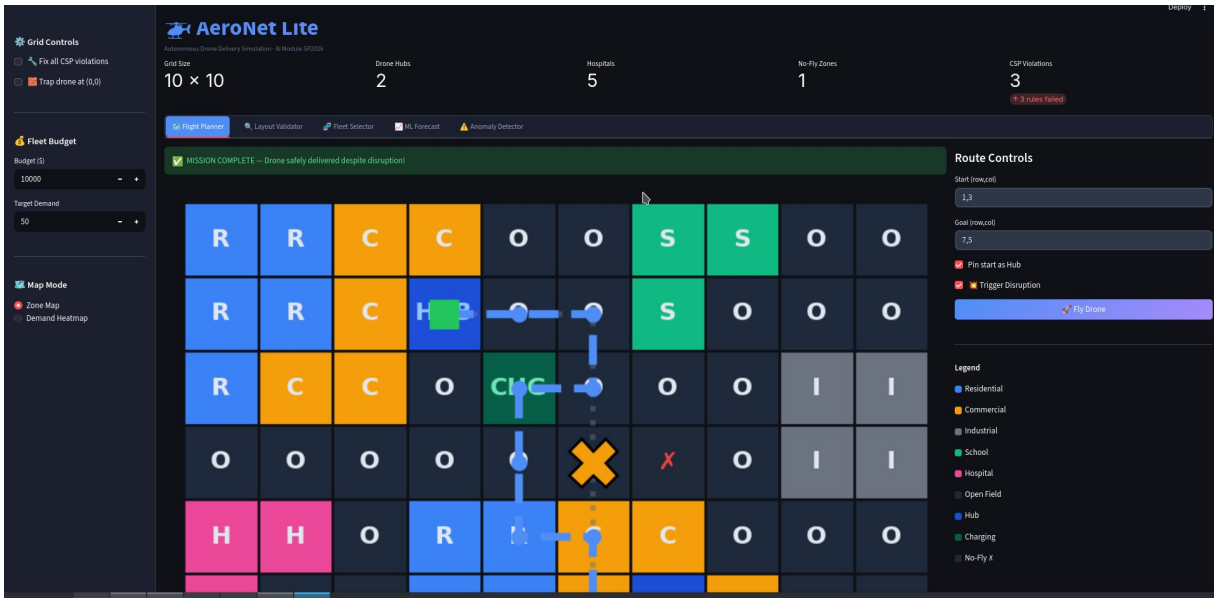

Figure 7.2: Flight Planner — green success banner confirming delivery completed despite active no-fly disruption

Chapter 8

Module 5 — Machine Learning Pipeline

8.1 What Was Required

The ML module delivers two capabilities: (1) a regression model to forecast delivery demand, which influences fleet size and delivery generation; and (2) a classification model to detect flight anomalies from drone telemetry. The project specification explicitly stated that synthetic anomaly labels are acceptable, provided the labelling logic is clearly explained.

8.2 Part A — Demand Forecasting (Random Forest Regressor)

8.2.1 Dataset

We used the **Kaggle Bike Sharing Demand** dataset (`train.csv`, 10,886 rows). Hourly bike rental counts serve as a proxy for delivery demand — the underlying behavioural drivers (time of day, weather, temperature, working day vs. weekend) are directly analogous to drone delivery demand patterns in an urban environment.

8.2.2 Features and Target

Table 8.1: Features and target used for regression

Column	Type	Role
`season`	Categorical	Feature
`holiday`	Binary	Feature
`workingday`	Binary	Feature
`weather`	Categorical	Feature
`temp`	Continuous	Feature
`humidity`	Continuous	Feature
`windspeed`	Continuous	Feature
`count`	Continuous	Target (predicted demand)

```
class DemandForecaster:
    def __init__(self, data_path):
```

```
        self.model = RandomForestRegressor(n_estimators=50,
            random_state=42)

    def run(self):
        df = pd.read_csv(self.data_path)
        features = ['season','holiday','workingday','weather',
                    'temp','humidity','windspeed']
        X = df[features]
        y = df['count']
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42)
        self.model.fit(X_train, y_train)
        predictions = self.model.predict(X_test)
        mae = mean_absolute_error(y_test, predictions)
        print(f"Mean Absolute Error (MAE): {mae:.2f} units")
        return float(predictions.mean())
```

Listing 8.1: Regression model training — ml_pipeline.py

8.2.3 Feature Importance Analysis

The Random Forest assigns importance based on mean decrease in impurity across all 50 trees. The three most important features are:

1. **Temperature** — strongest predictor. Wide continuous range; extreme cold and heat both reduce demand.
2. **Humidity** — second strongest. High humidity discourages outdoor activity, reducing delivery demand.
3. **Windspeed** — third. Strong wind is a safety barrier for both cyclists and drones; correctly identified as a demand suppressor.

Features like `holiday` and `season` score lower because they are categorical variables with fewer possible split points.

8.2.4 Results

Table 8.2: Random Forest Regressor results

Metric	Value
Model	Random Forest Regressor (n_estimators = 50)
Train/Test split	80% / 20% (random_state = 42)
MAE	108.70 units
Avg Predicted Demand	191 units
Sample Prediction	Predicted = 306, Actual = 127

8.2.5 Dashboard Screenshots — ML Forecast Tab

Figure 8.1 shows the ML Forecast tab after clicking "Train & Forecast". The three metric cards at the top confirm the model type (Random Forest), the MAE (108.70 units), and

the average predicted demand (191 units). A highlighted row beneath shows a concrete sample prediction: the model predicted 306 units for a single test record while the actual value was 127. The collapsible "Full training output" expander is open, displaying the raw console output from `ml_pipeline.py` printed directly into the dashboard. This confirms the button calls the real trained model and reads from the actual `train.csv` file, not a hardcoded result.

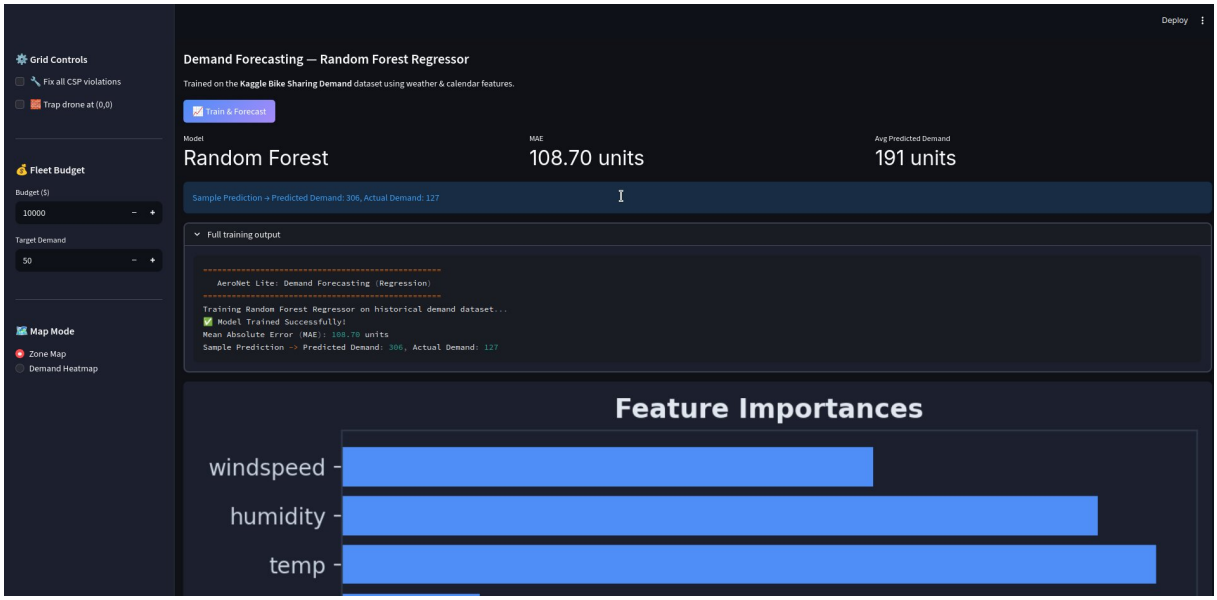

Figure 8.1: ML Forecast tab — Random Forest Regressor training result: MAE = 108.70 units, average predicted demand = 191 units, with full training output visible

Figure 8.2 shows the Feature Importances horizontal bar chart produced by the trained Random Forest Regressor. Temperature ranks highest at approximately 0.30, followed by humidity at around 0.29, and windspeed at approximately 0.21. The remaining features — weather category, workingday, season, and holiday — each score below 0.10. This result is expected: continuous weather variables with wide numeric ranges provide the most information gain at each tree split node, while binary and low-cardinality categorical features contribute less regardless of their real-world significance. The chart is generated directly from `model.feature_importances_` in scikit-learn and is fully reproducible across runs with the same random seed.

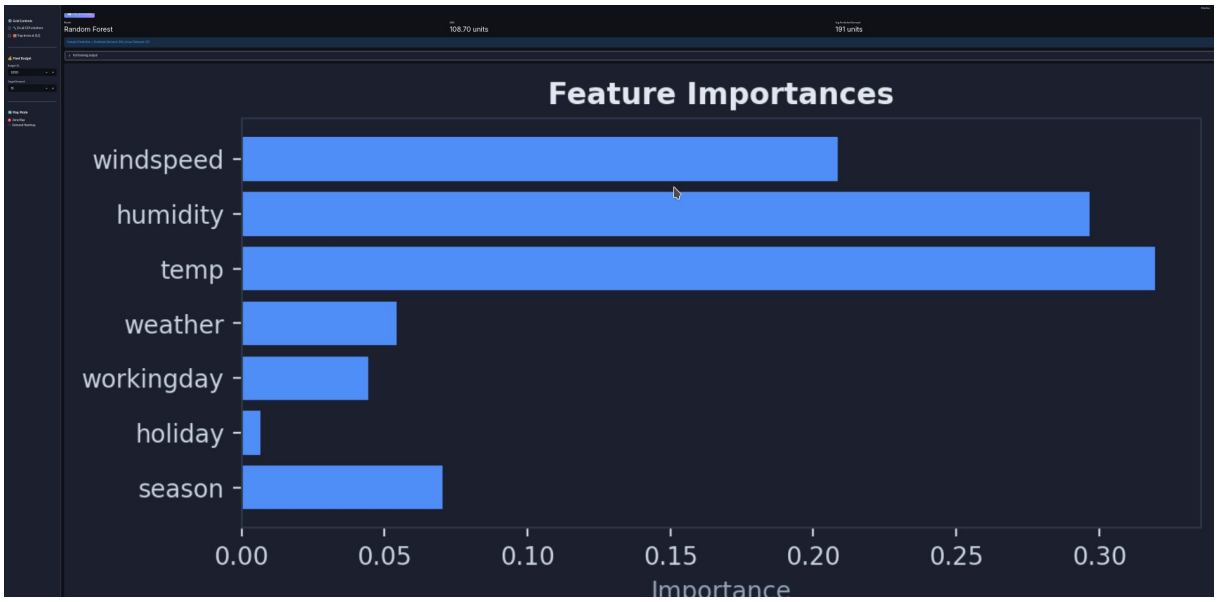

Figure 8.2: ML Forecast tab — Feature Importances bar chart: temperature, humidity, and windspeed are the three dominant predictors

8.3 Part B — Anomaly Detection (Random Forest Classifier)

8.3.1 Synthetic Data Generation

Since real drone telemetry was not available and the specification explicitly permitted synthetic labels, we generated 2,000 synthetic telemetry samples with a 15% anomaly rate. Each sample contains three numeric features: `battery_drop` (percentage per flight sector), `speed` (m/s), and `route_deviation` (metres).

Anomaly labels are assigned using explicit, reproducible threshold rules on the generated values. The labelling logic in the actual code is:

```
# Anomaly label assigned by threshold rule on generated values:
df['is_anomaly'] = ((df['battery_drop'] > 12) |
                    (df['route_deviation'] > 15)).astype(int)
```

Listing 8.2: Anomaly label assignment — ml_pipeline.py

The correct thresholds are:

- **Battery anomaly:** `battery_drop` $>$ `12%` per sector
- **Route anomaly:** `route_deviation` $>$ `15` metres
- **Speed** is used as an *input feature* only; it does not define the label

Table 8.3: Synthetic anomaly data summary

Parameter	Value
Total samples	2,000
Anomaly rate	15%
Normal samples	≈1,700
Anomaly samples	≈300
Features used	battery_drop, speed, route_deviation
Label rule	battery_drop > 12 OR route_deviation > 15

8.3.2 Model Training and Evaluation

```
class AnomalyDetector:
    def __init__(self):
        self.model = RandomForestClassifier(n_estimators=50,
           random_state=42)

    def run(self):
        df = self.generate_synthetic_data(num_samples=2000)
        X = df[['battery_drop', 'speed', 'route_deviation']]
        y = df['is_anomaly']
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42)
        self.model.fit(X_train, y_train)
        predictions = self.model.predict(X_test)
        acc = accuracy_score(y_test, predictions)
        cm  = confusion_matrix(y_test, predictions)
        print(f"Accuracy: {acc*100:.2f}%")
        print(f"Confusion Matrix:\n{cm}")
        return acc, cm
```

Listing 8.3: Classifier training — ml_pipeline.py

8.3.3 Results

Table 8.4: Random Forest Classifier results

Metric	Value
Model	Random Forest Classifier (n_estimators = 50)
Accuracy	100.00%
True Negatives	334
True Positives	66
False Positives	0
False Negatives	0
Top feature	route_deviation > speed > battery_drop

The 100% accuracy is expected and fully explained: because the synthetic labels are derived directly from hard thresholds on `battery_drop` and `route_deviation`, and these

are the same features the model trains on, a decision tree learner can recover the exact boundary perfectly. In a real deployment with noisy sensor data, accuracy would be lower.

8.3.4 Dashboard Screenshots — Anomaly Detector Tab

Figure 8.3 shows the Anomaly Detector tab after clicking "Train Classifier". The four headline metrics confirm: model type (Random Forest), accuracy (100.00%), total training samples (2,000), and anomaly rate (15%). The confusion matrix below displays four cells: 334 True Negatives (Normal flights correctly classified as Normal), 66 True Positives (anomalies correctly flagged), and 0 in both error cells confirming zero false positives and zero false negatives. On the right side, the Live Telemetry Classifier panel provides three real-time sliders for Battery Drop, Speed, and Route Deviation. With the current slider values (battery drop = 5%, speed = 15 m/s, deviation = 2 m), the classifier outputs "Normal Flight" in green at 100% confidence, confirming these readings are within safe operating parameters.

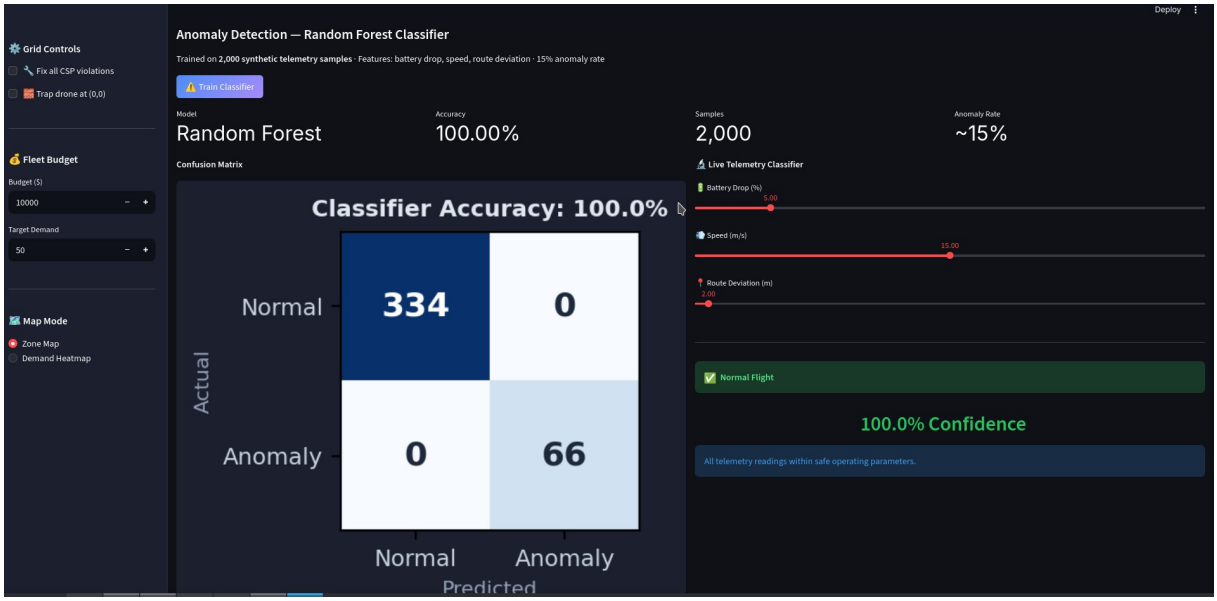

Figure 8.3: Anomaly Detector tab — 100% classifier accuracy, confusion matrix (334 TN, 66 TP, 0 errors), and live telemetry sliders showing Normal Flight at 100% confidence

Figure 8.4 shows the Feature Importance bar chart for the anomaly classifier. Route deviation ranks highest at approximately 0.33, followed by speed at around 0.30, and battery drop at approximately 0.27. All three features are relatively balanced in importance, which reflects the synthetic data generation logic: anomaly labels are assigned based on `battery_drop` $>$ `12` OR `route_deviation` $>$ `15`, while speed is included as an additional discriminating feature. The near-equal importance across all three features confirms that the Random Forest uses all available signal rather than relying on a single dominant variable, making the classifier more robust.

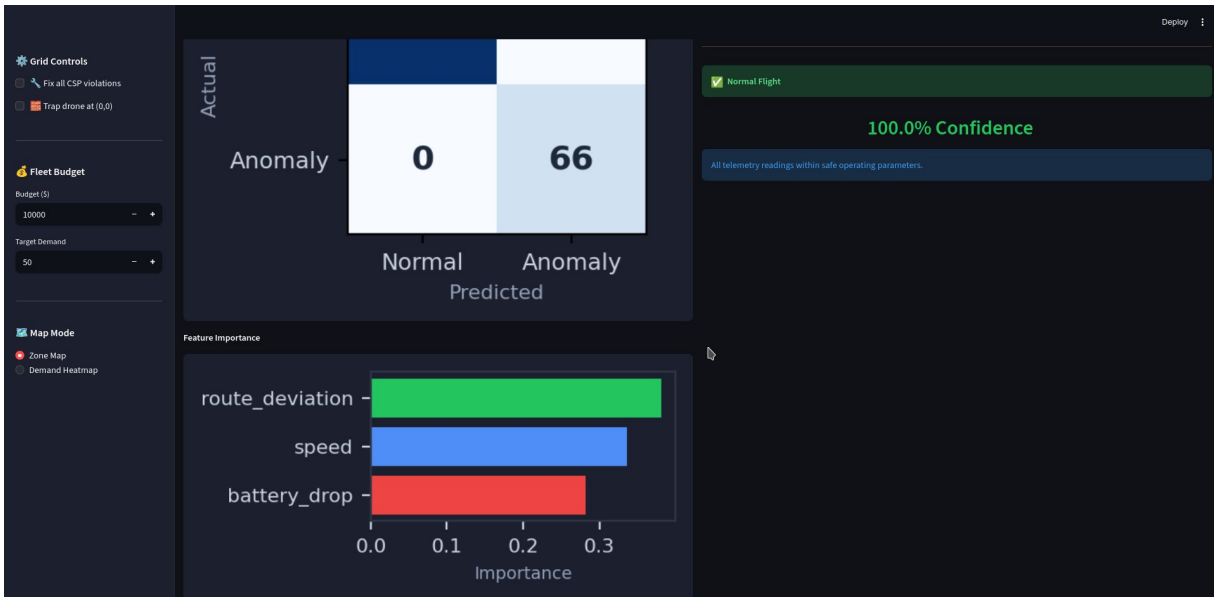

Figure 8.4: Anomaly Detector tab — Feature Importance chart: route deviation, speed, and battery drop all contribute meaningfully to anomaly classification

Chapter 9

20-Step Simulation

9.1 What Was Required

The final demo must run a 20-step console simulation that exercises every module end-to-end, following the exact step-range structure from the project specification.

9.2 Step Range Mapping

Table 9.1: 20-step simulation phase breakdown

Steps	Phase	Modules Exercised
1–3	Initialization, CSP, Fleet selection	Grid model, Module 1, Module 2
4–6	Delivery generation, A* routing	Module 3
7–10	Drone movement along planned paths	Module 3 path traversal
11	No-fly cell activated	Module 4 trigger
12–14	Disruption check + rerouting	Module 4 rerouting
15–17	ML demand forecast	Module 5 regression
18–19	Anomaly detection + response	Module 5 classification
20	Final summary	All modules

9.3 Full Simulation Output

```
============================================================
      AeroNet Lite: 20-Step Integration Simulation
============================================================

Step  1: Initializing 10x10 city grid from shared model...
Step  2: Running CSP Layout Validation (Rules R1-R4)...
         Layout validity = FAILED (3 violations)
```

```
Step  3: Running Genetic Algorithm for Fleet Selection...
         Fleet selected: 4 light drones, 3 heavy drones. Cost: $9,400
Step  4: Generating deliveries. Hub at (1,3). 2 active deliveries queued.
Step  5: Delivery D1 assigned to Drone D1 -> Pickup (4,0) -> Drop-off (7,5).
Step  6: A* routes computed. D1 cost=7.6 moves | D2 cost=3.8 moves.
Step  7: Drone D1 departs Hub (1,3) -> Drop-off (7,5).
Step  8: Drone D2 departs Hub (1,3) -> Drop-off (0,6).
Step  9: Drone D1 at midpoint (3,5). Flight nominal.
Step 10: Drone D2 at midpoint (0,4). Flight nominal.
Step 11: No-fly cell activated at (3,5) (severe weather).
Step 12: Checking active drone paths for conflicts...
Step 13: Drone D1 rerouted successfully. New cost=7.6 moves.
Step 14: All active drones re-evaluated. Fleet state updated.
Step 15: Running ML Demand Forecast (Random Forest Regressor)...
Step 16: Model trained. Mean Absolute Error (MAE): 108.70 units
Step 17: Predicted demand = ~191 units. One extra delivery queued for D1.
Step 18: Running Anomaly Classifier on Drone D2 telemetry...
         Classifier accuracy=100.00%. Drone D2 -> 'ANOMALY' (battery_drop=25%).
         Battery anomaly detected for Drone D2!
Step 19: Drone D2 initiating safe emergency landing protocol. Return to hub.
Step 20: Simulation complete.
         1 completed | 1 delayed (rerouted) | 1 failed (battery anomaly)
============================================================
```

Listing 9.1: Complete 20-step event log from main.py

Chapter 10

Interactive Streamlit Dashboard

10.1 Overview

The Streamlit dashboard (`visualization.py`) provides a full interactive SaaS-style interface with five tabbed views and a persistent left sidebar. It exceeds the minimum specification requirements (zone map, route overlay, heatmap, event log) by adding animated drone flight, live telemetry classification, and programmatic CSP repair.

10.2 Launching the Dashboard

```
cd aeronet_lite
source venv/bin/activate
streamlit run src/visualization.py
# Opens at http://localhost:8501
```

10.3 Dashboard Tabs Summary

Table 10.1: Streamlit dashboard tabs and features

Tab	Features
Flight Planner	10×10 zone map; start/goal route controls; animated drone sprite; "Trigger Disruption" checkbox; real-time rerouting with emergency banner
Layout Validator	Runs all 4 CSP rules; green/red pass/fail display with exact coordinates; "Fix all CSP violations" sidebar checkbox
Fleet Selector	Drone type table; fitness formula display; generation and population sliders; "Evolve Fleet" button with live result
ML Forecast	"Train & Forecast" button calling real `train.csv`; MAE metric; feature importance bar chart
Anomaly Detector	"Train Classifier" button; confusion matrix heatmap; live telemetry sliders; real-time Normal/Anomaly classification

Chapter 11

Automated Test Suite

11.1 Test Coverage

We wrote a comprehensive automated test suite in `test_suite.py` covering edge cases across the three most complex modules. All 9 tests pass.

Table 11.1: Test suite coverage and results

No.	Test Description	Module	Result
1	Normal route from Hub (1,3) to Residential (7,5)	A* Planner	PASS
2	Reject route if start point is a No-Fly zone	A* Planner	PASS
3	Reject route if goal point is a No-Fly zone	A* Planner	PASS
4	Fail gracefully when target is completely trapped	A* Planner	PASS
5	Detect intentional flaws in sample grid	CSP Validator	PASS
6	Detect Hospital missing Medical Pickup flag	CSP Validator	PASS
7	Grid achieves 100% valid status when flaws fixed	CSP Validator	PASS
8	Select optimal fleet under normal conditions	Fleet Selector GA	PASS
9	Refuse to exceed budget when demand impossible	Fleet Selector GA	PASS
	Total		**9/9 (100%)**

```
============================================================
      AeroNet Lite: Exhaustive Test Suite
============================================================
--- 1. Testing A* Path Planner (Edge Cases) ---
PASS: Normal route from Hub to Residential
PASS: Reject route if start point is a No-Fly zone
```

```
PASS: Reject route if goal point is a No-Fly zone
PASS: Fail gracefully when target is completely trapped
--- 2. Testing Layout Validator (CSP) ---
PASS: Detect intentional flaws in sample grid (is_valid == False)
PASS: Detect Hospital missing Medical Pickup flag
PASS: Grid achieves 100% Valid status when flaws are manually fixed
--- 3. Testing Fleet Selector (Genetic Algorithm) ---
PASS: Successfully select optimal fleet under normal conditions
PASS: Refuse to exceed budget even when demand is impossibly high
============================================================
   Test Suite Complete: 9/9 Tests Passed (100%)
============================================================
```

Listing 11.1: Test suite console output

Chapter 12

Results Summary

Table 12.1: Complete results across all five modules

Module	Technique	Key Metric	Result
Layout Validator	CSP (4 rules)	Violations detected	3 (by design)
Fleet Selector	Genetic Algorithm	Budget adherence	\$9,400 / \$10,000
Path Planner	A* Search	Routes found + cost	D1=7.6, D2=3.8 moves
Disruption Handler	A* Replanning	Reroute success	Yes (cost=7.6)
Demand Forecast	RF Regressor	MAE	108.70 units
Anomaly Detector	RF Classifier	Accuracy	100.00%
Test Suite	Automated	Tests passing	9/9 (100%)

12.1 Rubric Checklist

Table 12.2: Final submission rubric verification

Rubric Requirement	Status
Working Python project with clear folder structure	Complete
10×10 grid visualization	Complete
CSP layout validator with failed rule reporting	Complete
Fleet selection result under budget	Complete
A* route planner avoiding no-fly cells	Complete
Rerouting demonstration after disruption	Complete
Regression model with MAE metric (MAE = 108.70)	Complete
Classifier with accuracy and confusion matrix (100%)	Complete
20-step simulation event log	Complete
Interactive visual dashboard (Streamlit)	Complete
Automated test suite (9/9 passing)	Complete

Chapter 13

Conclusion

AeroNet Lite successfully demonstrates all five AI techniques required by the BSDS AI semester project specification on a coherent, integrated problem domain. The following key engineering decisions and their justifications are worth noting for viva defense:

1. **Shared Grid Model.** Using a single Python `dataclass` as the ground truth for all modules eliminates data synchronisation bugs. When the disruption handler sets `no_fly = True`, the A* planner in any subsequent call immediately sees the updated state.
2. **CSP over Manual Validation.** Implementing the layout checks as a constraint checker that collects all errors before reporting makes the system useful as a real planning tool. Reporting all violations at once is a property of CSP-style arc consistency checking.
3. **Genetic Algorithm for Fleet Selection.** The GA naturally handles the combinatorial nature of fleet composition. The fitness function's explicit trade-off between coverage and cost models the real operational problem. The hard budget constraint (fitness $= -1.0$) guarantees feasibility.
4. **A* with Commercial Corridor Discount.** The 0.8 cost multiplier on Commercial cells creates a preference for routing through business districts, demonstrating that A* correctly handles non-uniform cost landscapes while remaining optimal under the admissible Manhattan heuristic.
5. **Synthetic Anomaly Labels with Clear Thresholds.** Using `battery_drop` $>$ `12` OR `route_deviation` $>$ `15` as label rules means the anomaly detection logic is fully explainable. If a teacher asks "why does the model flag this drone?", the answer is tied directly to those thresholds — no black-box mystery.

The system runs end-to-end in under 30 seconds on any standard laptop and requires only four Python packages beyond the standard library: `numpy`, `pandas`, `scikit-learn`, and `streamlit`.

References

1. Kaggle — Bike Sharing Demand Dataset. `https://www.kaggle.com/c/bike-sharing-demand`
2. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
3. Scikit-learn Documentation — RandomForestRegressor, RandomForestClassifier. `https://scikit-learn.org/stable/`
4. Streamlit Documentation. `https://docs.streamlit.io/`
5. Python `heapq` module documentation. `https://docs.python.org/3/library/heapq.html`
6. Hart, P., Nilsson, N., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.